

Python - Conditions - If->Else

אחיהוד

Conditions - If->Else

```
age = 15
if age ≥ 18:
    print("you are an adult")
else:
    print("you are a minor")
```

תנאים ב-Python

If/Else

אם זה נכון, עשה A ,

אחרת, עשה B .

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python conditions.py
you are a minor
```

Exercise

תרגיל if

1. כתבו סקריפט שבודק אם $10 + 10$ שווה ל-20 .

אם שווה ל-20, הדפיסו : "the number is 20"

אם לא שווה ל-20, הדפיסו : "the number is not 20"

```
num1 = 40
num2 = 45
sum = num1 + num2

if sum == 85:
    print("success")
else:
    print("failure")
```

דוגמא

```
num1 = 10
num2 = 10
sum = num1 + num2

if sum == 20:
    print("the number is 20")
else:
    print("the number is not 20")
```

פתרון

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python conditions.py
the number is 20
```

Python - Conditions - If->Else

אחיהוד

```
username = "admin"
password = "1234"

if username == "admin" and password == "1234":
    print("login successful")
else:
    print("login faied")
```

תנאים ב-Python

If/Else

אם זה נכון, עשה A ,
אחרת, עשה B .
לוגיקה של מסך לוגין.

Exercise - if->else

תרגיל if

1. כתבו סקריפט למסך לוגין, שבודק אם גם מייל וגם סיסמה נכונים. השתמשו ב-print כדי להדפיס אם נכון/לא נכון.
2. כתבו סקריפט שבודק טמפרטורה (temperature) . אם הטמפרטורה גדולה/שווה ל-30, הדפיסו שחם היום. אם מתחת ל-20, הדפיסו שקר היום.

```
user_email = "anna@gmail.com"
user_password = "1234"

if user_email == "anna@gmail.com" and user_password == "1234":
    print("login successful")
else:
    print("login faied")
```

פתרון

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python conditions.py
login successful
```

```
temperature = 30
if temperature ≥ 30 :
    print("it_is_hot_today")

elif temperature < 20 :
    print("it_is_cold_today")

else :
    print("undefind")
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python conditions.py
it_is_hot_today
```

Python – Conditions – If->Else

אחיהוד

Conditions-if->elif->else

```
score = 75
if score ≥ 90 :
    print("Grade: A")
elif score ≥ 70:
    print("Grade: B")
elif score ≥ 50:
    print("Grade: C")
else:
    print("Grade: F")
```

תנאים ב-Python

If/elif/else

- אם זה נכון, עשה A .
- אם לא, עשה B .
- או לא, עשה C .
- אחרת, עשה D .

Exercise – if->else

תרגיל if

- כתבו סקריפט שבודק אם username גדול/שווה מ-6 וקטן מ-32 .
 - אם גדול מ-32 , הדפיסו שגיאה מתאימה .
 - אם קטן מ-6 , הדפיסו שגיאה מתאימה.
 - אם נמצא בטווח הנכון, הדפיסו : "Success".
- צרו משתנה בשם weather וערכו : "sunny".
 - כתבו סקריפט שמסביר עם מה לצאת מהבית כש:
 - א. weather == "sunny"
 - ב. weather == "rainy"
 - ג. weather == "snowy"
 - ד. מזג אוויר נהדר, לא צריך משהו מיוחד.

```
username = 19
if username > 32:
    print("login faied")
elif username < 6:
    print("login faied")
else:
    print("Success")
```

פתרון

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python conditions.py
Success
```

Python - Conditions - If->Else

אחיהוד

```
weather = "sunny"
if weather == "sunny":
    print("wear summer clothes")
elif weather == "rainy":
    print("wear winter clothes and take an umbrella with you")
elif weather == "snowy":
    print("wear Thermal clothes")
else:
    print("Great weather, nothing special needed")
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python conditions.py
wear summer clothes
```

Exercise - if->else

בודקים האם משתמש הצליח להתחבר למערכת.

באוטומציה, כשהתחברות מצליחה, השרת מחזיר קוד סטטוס מספרי, קוד 200 אומר שההתחברות עברה בהצלחה.

נתון המשתנה הבא שמייצג את התשובה מהשרת: `status_code = 200`

לכתוב קוד מבוסס על if ו-else אשר בודק:

אם הערך של `status_code` שווה ל-200, הקוד צריך להדפיס למסך: "Login Successful"

אחרת (בכל מצב אחר), הקוד צריך להדפיס למסך "Login Failed".

פתרון

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python conditions.py
Login Successful
```

```
status_code = 200
if status_code == 200:
    print("Login Successful")
else:
    print("Login Failed")
```

תרחיש לראיון עבודה

את מפתחת אוטומציה שכותבת טסט למערכת קופונים של חנות אינטרנטית.

הכלל של החנות: זכאות משלוח חינם – סכום קנייה 150 שח ומעלה.

המראיין נותן את המשתנה הבא, שמייצג את סכום הקנייה הנוכחי של הלקוח, ומבקש לבדוק את הלוגיקה באמצעות if ו-else בלבד:

`cart_total = 120`

Python – Conditions – If->Else

אחיהוד

לכתוב קוד שיבדוק את סכום הקנייה .

אם סכום הקנייה גדול או שווה ל-150, הקוד צריך להדפיס למסך: "Free Shipping"

אחרת (בכל מצב אחר), הקוד צריך להדפיס למסך: "Shipping Cost: 30"

פתרון

```
cart_total = 120
if cart_total >= 150:
    print("Free Shipping")
else:
    print("Shipping Cost: 30")
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python conditions.py
Shipping Cost: 30
```

הרחקה מהשוליים/הזחה - *Indentation*

אחיזה

הרחקה מהשוליים היא איך שפיתון מבין את הקוד .

פיתון יודע שה-`print` שייך ל-`if` ספציפי.

הוא עושה זאת באמצעות `indentation` .

ה-indentation ילווה אותנו תמיד בכתיבת קוד.

```
age = 15
if age ≥ 18:
    print("you are an adult")
else:
    print("you are a minor")
```

Syntax And Logic – If->Elif->Else

אחידות

Syntax

נכתוב את מילת התנאי קודם – "if" ואז את המשתנה עליו אנחנו מבצעים את ה-If.

נשתמש בסימן "==" אם נרצה לבדוק שהמשתנה שווה לערך כלשהו.
נשתמש באופרטורים, כמו: "<=", ">=", "==" כשנרצה לבדוק אם הערך קטן/גדול מ, או שווה ל ...

כשנסיים לכתוב את התנאי במלואו, נשים נקודתיים.
מה שבא אחרי הנקודתיים זה ה-"עשה את זה" שלנו.

```
syntax = True  
if syntax == True:  
    print("syntax is true")
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python conditions.py  
syntax is true  
_
```

כשמקבלים משתנים ממקור חיצוני ולא בטוחים מה סוג הנתונים שלהם, פייתון יצרה את הפונקציה המובנית `type()`.

כדי לראות את התוצאה בטרמינל, אנחנו פשוט עוטפים אותה ב-`print`

```
print(type(syntax))
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python conditions.py  
syntax is true  
<class 'bool'>  
_
```

Logic

חייב ליצור משתנים לפני שאנחנו כותבים את התנאים.
If הוא הראשון, אחכ (אופציונלי) יבוא כמה elif שצריך ואז (אופציונלי) else.

Syntax And Logic – If->Elif->Else

אחידוד

```
variable1 = 12
variable2 = 24

if variable1 > 10 and variable2 == 22:
    print("success")
elif variable1 >= 10 and variable2 >= 22:
    print("success")
else:
    print("failure")
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python conditions.py
success
```

מה זה בעצם elif

המילה elif היא פשוט קיצור של שתי מילים באנגלית Else If (ואם לא, אז האם...).

אם If

השאלה הראשונה שאנחנו שואלים (למשל: "האם יש לי באג חמור?")

else אחרת

ברירת המחדל שלנו.

פח האשפה של התנאים.

מה קורה אם התשובה ל- if הייתה "לא", (למשל: "else" - הכל תקין).

מה קורה כשהתשובה היא לא רק כן ולא, מה קורה אם יש לנו יותר משתי אפשרויות, פה נכנס ה- elif.

ה- elif הוא השאלה השנייה (או השלישית, או העשירית) שאנחנו שואלים, רק במידה והשאלה הראשונה (if) קיבלה את התשובה False (לא נכון)

מתי משתמשים ב-elif

משתמשים ב- elif כשיש לנו מספר תרחישים שונים שסותרים אחד את השני, ואנחנו רוצים שהקוד יבדוק אותם אחד אחרי השני עד שהוא מוצא את התשובה הנכונה, ואז יעצור.

אפשר להשוות לרמזור:

Syntax And Logic – If->Elif->Else

אחידות

if הרמזור ירוק >- סע.

elif הרמזור כתום >- האט והתכונן לעצירה.

else <- תעצור (כי נשאר רק אדום).

ברגע שהקוד מוצא תנאי אחד שמתקיים (True), הוא מבצע את מה שכתוב מתחתיו, ומדלג על כל שאר התנאים.

דוגמאות של elif

נניח שכותבים אוטומציה שבודקת סטטוס קוד (Status Code) שחזר מהשרת אחרי פעולת התחברות.

יש לנו כמה תרחישים אפשריים, לא רק "עבר" או "נכשל":

```
status_code = 404

# השאלה הראשונה
if status_code == 200:
    print("Login Successful!")

# נעבור לשאלה השנייה, אם השאלה הראשונה הייתה False
elif status_code == 404:
    print("Error: Page Not Found!")

# שרק רוצים elif אפשר לשים כמה!
elif status_code == 500:
    print("Error: Server Crashed!")

# אם אף אחד מהתנאים למעלה לא התקיים, נגיע לברירת המחדל
else:
    print("Unknown Status Code!")
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python conditions.py
success
Error: Page Not Found!
```

Syntax And Logic – If->Elif->Else

אחיהוד

איך פייתון קורא את זה

1. הוא בודק את ה- if האם 404 שווה ל-200? התשובה היא **False** הוא מדלג על ההדפסה.
 2. הוא יורד ל- elif הראשון, האם 404 שווה ל-404? התשובה היא **True**!
 3. הוא מדפיס "Error: Page Not Found!", וברגע שהוא עשה את זה - הוא יוצא לגמרי מהבלוק ולא ממשיך לבדוק את השאר.
- ה- else למטה כבר לא מעניין אותו.

למה 404 הוא לא "לוגאין בהצלחה"

- לוגיקה של עולם התוכנה .
- כשבודקים אתר (כמו במובייל או בווז), השרת מחזיר מספרים שנקראים קודי סטטוס (Status Code) כדי להגיד מה קרה:
- 200** = הכל תקין! **הצלחה** (Login Successful) .
 - 404** = **שגיאה**: העמוד לא קיים (Page Not Found) .
 - 500** = **שגיאה**: השרת קרס (Server Crashed) .

למה צריך ELIF ולא רק IF ו-ELSE

- הלוגיקה הזאת כללית מדי .
- אם כותבים רק if/else, ברגע שתהיה שגיאה, המערכת תגיד "יש שגיאה".
- אבל בבדיקות אוטומציה, זה לא מספיק לדעת שיש שגיאה - רוצים לדעת איזו שגיאה בדיוק !
- האם העמוד לא קיים (404) או שהשרת קרס (500) .
- כאן בדיוק נכנס ה- elif (שזה קיצור של Else-If ואם לא, אז האם...)
- אפשר להשוות לתפריט במסעדה

אם משתמשים רק ב- if ו- else, זה כמו לשאול שאלה של "כן/לא"

- IF** האם אתה רוצה המבורגר?
- ELSE** אחרת אז קח סלט, יש רק 2 אפשרויות.

Syntax And Logic – If->Elif->Else

אחיהוד

אבל מה אם יש תפריט שלם לבדוק, פה חייבים את ה-elif

IF אם הסטטוס הוא 200 (אם כן, תדפיס הצלחה ותסיים).

ELIF ואם לא, אז האם ... הסטטוס הוא 404 (אם כן, תדפיס שהעמוד לא נמצא ותסיים).

ELIF ואם לא, אז האם ... הסטטוס הוא 500 (אם כן, תדפיס קריסה ותסיים).

ELSE אחרת אם זה אף אחד מהמספרים שבדקתי למעלה, תדפיס שמדובר בסטטוס לא מוכר.

הסבר

משתמשים ב- elif כשיש לנו יותר מ-2 אפשרויות שאנחנו רוצים לבדוק על אותו משתנה, ואנחנו רוצים שהמחשב יבדוק אותן אחת אחרי השנייה, עד שהוא מוצא את התשובה הנכונה.

באוטומציה אמיתית, אף פעם לא יודעים מראש מה תהיה התוצאה

במציאות, הקוד מבצע לחיצה על כפתור "התחבר" באפליקציה, מחכה שנייה, ושואל את המערכת, מה קיבלנו.

המערכת זורקת אליו נתון חי באותו רגע, ורק אז הקוד בודק אותו.

הסטט שכותבים חייב להיות מוכן לכל תרחיש אפשרי שהאפליקציה תזרוק עליו.

הדוגמה האמיתית, בדיקת הודעת שגיאה במסך התחברות

ניקח מקרה קלאסי של בדיקה ידנית שנהפוך לאוטומציה, אנחנו מנסים להתחבר למערכת עם סיסמה שגויה, וצריכים לוודא שקופצת הודעת השגיאה הנכונה.

באוטומציה, הקוד קורא את הטקסט שקפץ על המסך ושומר אותו במשתנה בשם popup_message (אנחנו לא יודעים מראש מה הטקסט הזה יהיה עד שהסטט לא ירוץ).

Syntax And Logic – If->Elif->Else

אחיהוד

עכשיו אנחנו בונים את הלוגיקה של הטסט שלנו (ה"מוח" שמקבל החלטות):

```
פיתון שומר פה את מה שהוא קרא מהמסך באותו רגע  
popup_message = ... (יכול להיות פה כל דבר)  
  
if popup_message == "Login Successful":  
    print("Test Failed! System allowed access with wrong password!")  
  
elif popup_message == "Incorrect Password":  
    print("Test Passed! The correct error message was shown.")  
  
elif popup_message == "System Error":  
    print("Test Failed! The system crashed during login!")  
  
else:  
    print("Test Failed! An unknown pop-up appeared!")
```

למה לכתוב עם elif, ולא רק עם if-else

כי if ו-else נותנים רק 2 מגירות.

מגירה לתנאי שהתקיים, ומגירה לכל השאר.

אבל אנחנו צריכים בדיקה עמוקה, לדעת את ה"למה" המדויק!

יש הבדל עצום בין באג של קריסת מערכת ("System Error") לבין באג של הודעה לא מוכרת.

אנחנו צריכים 4 מגירות שונות כדי לדעת במדויק את סיבת הקריסה.

ה-elif מאפשר לנו להוסיף כמה מגירות שרק נרצה באמצע!

למה לא לכתוב הרבה שורות של if נפרדות בלי elif

אם נכתוב 4 שורות של if אחת מתחת לשנייה (בלי הקישור של ה-elif), פיתון יתנהג כמו רובוט טיפש, שיבדוק את השאלה הראשונה.

נניח שההודעה היא "Incorrect Password", הוא ידפיס "Test Passed" אבל במקום לעצור פה (כי מצאנו את התשובה), הוא ימשיך לבדוק סתם את כל שאר השורות של ה-if למטה, למרות שזה כבר לא רלוונטי.

ה-elif עושה שני דברים קריטיים

הוא מקשר את כל השאלות לשרשרת לוגית אחת.

ברגע שהוא מוצא תנאי אחד שמתקיים (True), הוא בורח החוצה ולא בודק יותר את שאר השורות.

Syntax And Logic – If->Elif->Else

אחיהוד

באוטומציה של אלפי טסטים, זה חוסך המון זמן ריצה לשרת.

הסבר

משתמשים ב- `elif` כשהטסט שלנו צריך להיות מוכן ליותר משתי תוצאות אפשריות, אנחנו רוצים שהוא יבדוק אותן אחת אחרי השנייה ויעצור ברגע שהוא מצא את הבינוגו.

בעולם הבדיקות יש גם בדיקות שהן שחור ולבן (`True / False`) לחלק מהרבה בדיקות שעושים ביום יום, אין יותר משתי אופציות, זה או "`כן`" או "`לא`".

דוגמה

כותבים סקריפט שבודק אם כפתור ה- "`Login`" מופיע על המסך, המערכת מחזירה תשובה בוליאנית, הכפתור שם (`True`) או לא שם (`False`).

במקרה כזה, אין מה לשאול שאלות המשך.

If האם הכפתור מופיע - < הטסט **עבר**.

Else אחרת (הוא לא מופיע) - < הטסט נכשל, זרוק שגיאה שהכפתור חסר.

אין פה מקום ל- `elif` כי אין אופציה שלישית לבדוק!

סיבה 2 (והכי חשובה), לפעמים אנחנו לא רוצים שהקוד יעצור

ה- `elif` עובד חכם כי ברגע שהוא מוצא את התשובה הנכונה, הוא נעצר ויוצא החוצה.

לא תמיד רוצים לעצור אחרי שמוצאים דבר אחד תקין.

למשל תרחיש של בדיקת שפיות (`Sanity Test`) לעמוד הבית, נניח שנדרש לבדוק 3 דברים בלתי תלויים בעמוד הבית של קופת חולים מכבי:

1. האם הלוגו מופיע?
2. האם השם של המשתמש כתוב נכון למעלה?
3. האם כפתור "קביעת תור" לחיץ?

Syntax And Logic – If->Elif->Else

אחידות

השרשרת תעצור אם נשתמש פה ב- elif

```
if logo_exists:  
    print("Logo is fine")  
elif user_name_correct:  
    print("Name is fine")  
elif button_clickable:  
    print("Button is fine")
```

הקוד יבדוק את הלוגו.

יראה שהוא שם (True), הוא ידפיס "Logo is fine", ויעצור לגמרי!
הוא בכלל לא יטרח לבדוק אם השם נכון או אם הכפתור לחיצה, כי אמרנו לו שאלו שאלות של "ואם לא, אז..."

במידה ומשתמשים רק ב- if נפרדים (בדיקות בלתי תלויות)

במקרה כזה, אנחנו נכתוב כמה שורות if נפרדות, שלא קשורות אחת לשנייה בכלל, בלי elif ובלי else.

```
if logo_exists:  
    print("Logo is fine")  
  
if user_name_correct:  
    print("Name is fine")  
  
if button_clickable:  
    print("Button is fine")
```

פה, פייתון הוא עובד חרוץ שלא עוצר.

הוא יבדוק את הלוגו (וידפיס), וגם יבדוק את השם, וגם יבדוק את הכפתור.

עשינו בדיקה מלאה ומקיפה של העמוד, בלי שהקוד "יברח" לנו באמצע.

לסיכום (חוק הברזל לאוטומציה):

If – else בלבד, כשיש לנו שאלות של "כן או לא" (כפתור קיים או לא קיים).

if , elif , else, כשיש לנו משתנה אחד שיכולות להיות לו הרבה תוצאות שונות, ואנחנו צריכים לתפוס בדיוק אחת מהן ולעצור (כמו סוגי השגיאות 404, 500, 200).

Syntax And Logic – If->Elif->Else

אחידות

כמה **if** נפרדים (ללא **elif**), כשאנחנו רוצים לעבור על צ'קליסט של בדיקות ולוודא שהכל הכל עובד, מבלי לעצור באמצע.

Edge Cases הלא צפויים

בדיוק בגלל שאנחנו בני אדם שמפספסים או ממהרים, אנחנו בונים את המכונה (האוטומציה) שתעשה את העבודה השחורה והמדויקת בשבילנו.

איך אנחנו מגנים על עצמנו בקוד בעזרת תכנון חכם, כאן בדיוק נכנס הכוח האמיתי של ה- **else** (ברירת המחדל).

אפשר להשוות את ה- **else** ל"רשת הביטחון" שלנו .

נניח שכיסינו בעזרת **elif** ארבעה תרחישים שונים שלדעתנו יכולים לקרות, אבל פספסנו את התרחיש החמישי .

אם בסוף כל השרשרת הזאת תמיד נכתוב **else** , זה אומר לפייתון שאם **התקבל משהו** לא צפוי ולא כתבנו לו חוק – הוא יקפיץ הערה אדומה כדי שנבדוק את זה.

```
else:  
    print("ALERT: Something completely unexpected happened! Go check the logs!")
```

גם אם התפספס משהו, ה- **else** יתפוס את זה ויציל מפספוס באג קריטי.

Syntax And Logic – If->Elif->Else

אחיזה

חשוב שנבין את הלוגיקה של ה-**if**

התנאי הראשון קובע מאיפה להתחיל.

שאר התנאים עובדים בצורה לוגית ולוקחים בחשבון את מה שכבר פולטר לפניהם .
כלומר, $age \leq 18$ מתחיל מאיפה ש- $age < 13$ הפסיק.

```
age = 16
if age < 13 :
    print("Child")

elif age ≤ 18 :
    print("Teen")

else :
    print("Adult")
```

1,2,3,4,5,6,7,8,9,10,11,12 ומטה

מה שנשאר לאחר התנאי הראשון:
13,14,15,16,17,18

מה שנשאר לאחר התנאי השני:
19 ומעלה

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python conditions.py
Teen
```